

Zero to Hero Study Handbook:

computer-use

Module 1: Foundations and Architecture

- What this project does: `computer-use` is a local full-stack workbench for provider-native Computer Use agents (Google Gemini, Anthropic Claude, OpenAI) running against a Dockerized Linux desktop.
- Primary use cases: browser automation, desktop app automation, screenshot-driven task execution, optional provider-native web-search planning, and optional document-grounded runs.

Core paradigms and patterns used in this repo

- Adapter pattern: provider differences are normalized behind shared interfaces in `backend/providers/*.py` and `backend/engine/*`.
- Orchestrator loop pattern: one session loop (`backend/loop.py::AgentLoop`) controls lifecycle, state transitions, callbacks, and termination.
- Event-driven architecture: backend broadcasts real-time events (`log`, `step`, `screenshot`, `agent_finished`) over `/ws`; frontend reacts via `useWebSocket`.
- Protocol translation layer: model tool actions are translated into desktop operations through `DesktopExecutor`, then sent to `docker/agent_service.py`.
- Schema-first validation: Pydantic request/response models in `backend/models/schemas.py` plus runtime checks in API handlers.
- Defense in depth: host allowlist, origin checks, optional shared token (`CUA_WS_TOKEN`), request size limits, action allowlists.

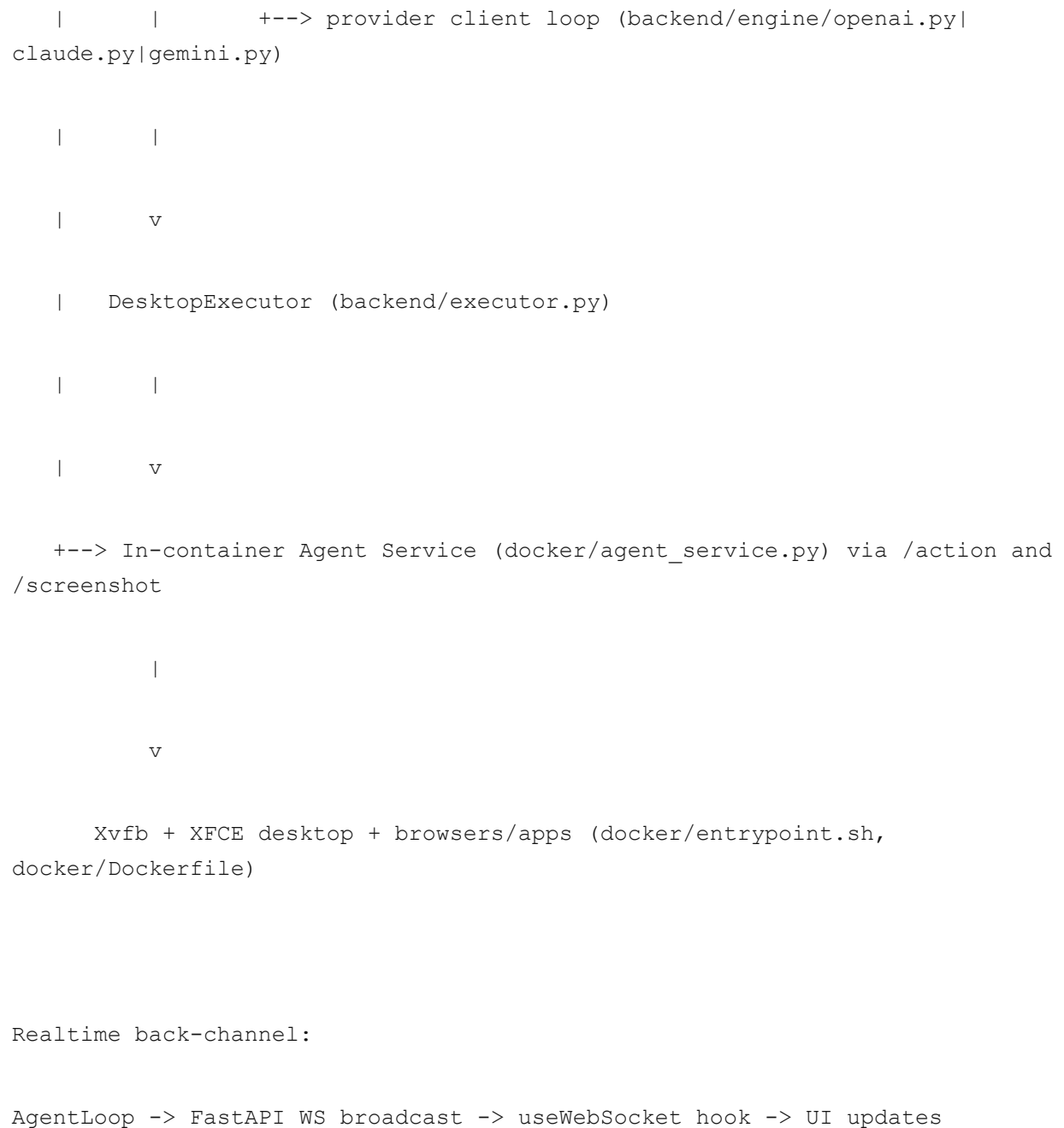
Architecture components and interactions

- Frontend: React + Vite SPA in `frontend/src/pages/WorkbenchPage.jsx`.
- Backend API: FastAPI app in `backend/server/__init__.py` exposing REST, WebSocket, and noVNC proxy routes.
- Session orchestrator: `AgentLoop` in `backend/loop.py`.
- Provider engine: `ComputerUseEngine` in `backend/engine/__init__.py` plus provider clients.
- Provider wrapper layer: `backend/providers/*` with uniform `run(...)` contracts.

- Desktop action executor: `backend/executor.py::DesktopExecutor`.
- Sandbox runtime: Docker container lifecycle in `backend/infra/docker.py`; in-container control plane in `docker/agent_service.py`.
- File pipeline: upload store in `backend/infra/storage.py` and provider-specific prep in `backend/files.py`.

Main flow diagram





Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions
backend/main.py	Backend entrypoint and Uvicorn launch	main(), _enforce_public_bind_guardrail()
backend/server/ __init__.py	FastAPI app,	api_start_agent(),

File/Directory Path	Primary Responsibility	Key Classes/Functions
	middleware, auth/origin guards, REST + WS endpoints	<code>websocket_endpoint()</code> , <code>vnc_ws_proxy()</code> , <code>_require_origin()</code> , <code>_require_rest_auth()</code>
<code>backend/models/schemas.py</code>	Typed API/session contracts	<code>StartTaskRequest</code> , <code>TaskStatusResponse</code> , <code>AgentSession</code> , <code>StepRecord</code> , <code>ActionType</code>
<code>backend/loop.py</code>	Session orchestration and callback wiring	<code>AgentLoop.run()</code> , <code>_run_computer_use_engine()</code> , <code>request_stop()</code>
<code>backend/engine/__init__.py</code>	Unified engine facade + provider construction	<code>ComputerUseEngine.execute_task()</code> , <code>validate_builtin_search_config()</code>
<code>backend/engine/openai.py</code>	OpenAI Computer Use client logic	<code>OpenAICUClient.run_loop()</code> , <code>_execute_openai_action()</code>
<code>backend/engine/claude.py</code>	Anthropic Computer Use client logic	<code>ClaudeCUClient.run_loop()</code> , <code>iter_turns()</code> , <code>build_web_search_tool()</code>
<code>backend/engine/gemini.py</code>	Gemini Computer Use client	<code>GeminiCUClient.run_loop()</code> , <code>iter_turns()</code>

File/Directory Path	Primary Responsibility	Key Classes/Functions
	logic	
backend/providers/____init____.py	Provider runtime dispatch	runner_for(), run_client()
backend/providers/_common.py	Shared provider utilities	stream_client_run_loop(), maybe_plan_with_web_search()
backend/providers/planner.py	Optional web-search planning phase	create_web_execution_brief(), , build_planned_computer_use_task()
backend/executor.py	Action translation into in-container API	DesktopExecutor.execute(), _post_action(), capture_screenshot()
backend/files.py	Provider-aware file attachment bridge	validate_attached_files(), prepare_openai_file_search(), , prepare_anthropic_documents()
backend/infra/storage.py	Upload persistence and lifecycle	FileStore.add_stream(), gc(), extract_text()
backend/infra/config.py	Environment config and key resolution	Config.from_env(), resolve_api_key(), get_all_key_statuses()
backend/infra/	Docker	start_container(),

File/Directory Path	Primary Responsibility	Key Classes/Functions
<code>docker.py</code>	build/start/stop/readiness	<code>_wait_for_service()</code> , <code>get_container_status()</code>
<code>backend/safety.py</code>	Safety confirmation handshake registry	<code>arm()</code> , <code>verify_nonce()</code> , <code>set_decision()</code> , <code>clear()</code>
<code>docker/agent_service.py</code>	In-container desktop control API	<code>AgentHandler.do_GET()</code> , <code>do_POST()</code> , <code>_dispatch_action()</code> , <code>_dispatch_desktop()</code>
<code>docker/entrypoint.sh</code>	Container desktop bootstrap sequence	Startup flow for Xvfb, XFCE, VNC, websockify, agent service
<code>frontend/src/pages/WorkbenchPage.jsx</code>	Main UI composition and handlers	<code>handleStart()</code> , <code>handleStartContainer()</code> , <code>handleValidateKey()</code>
<code>frontend/src/hooks/useSessionController.js</code>	Session lifecycle coordinator	<code>start()</code> , <code>stop()</code> , <code>finalizeAgentRun()</code>
<code>frontend/src/hooks/useWebSocket.js</code>	WS connection and stream handling	<code>connect()</code> , <code>setScreenshotMode()</code> , <code>isPlausibleWsMessage()</code>
<code>frontend/src/api.js</code>	REST client wrappers	<code>startAgent()</code> , <code>stopAgent()</code> , <code>uploadFile()</code> , <code>confirmSafety()</code>

File/Directory Path	Primary Responsibility	Key Classes/Functions
setup.sh	One-command bootstrap + launch	script flow
dev.py	Daily local orchestrator	<code>_compose_restart()</code> , <code>_spawn_services()</code> , <code>main()</code>
.env.example	Runtime env template	env keys for local config
docker-compose.yml	Local service definition	service cua-environment

Module 3: Core Execution Flows

Flow A: Start agent from UI to running session

1. `WorkbenchPage.handleStart()` builds payload from selected UI state (provider, model, task, maxSteps, optional reasoning/search/files).
2. `useSessionController.start()` calls `frontend/src/api.js::startAgent()`.
3. `startAgent()` sends `POST /api/agent/start` with backend field names (api_key, max_steps, execution_target, etc.).
4. `backend/server/__init__.py::api_start_agent()` validates:
5. `engine == "computer_use"`
6. `execution_target == "docker"`
7. provider/model against `backend/models/allowed_models.json`
8. max_steps hard cap
9. attachments via `backend/files.py::validate_attached_files()`
10. reasoning/search config via `validate_builtin_search_config()`
11. API key via `resolve_api_key()`
12. Backend starts/checks sandbox via `backend/infra/docker.py::start_container()`.

13. Backend creates `AgentLoop(...)`, stores it in `_active_loops`, and schedules `_run_and_notify()`.

14. API returns session handle.

Start request body shape:

```
{

  "task": "string",

  "api_key": "string|null",

  "model": "string",

  "max_steps": 50,

  "engine": "computer_use",

  "provider": "google|anthropic|openai",

  "execution_target": "docker",

  "reasoning_effort": "none|minimal|low|medium|high|xhigh",

  "use_built_in_search": true,

  "attached_files": ["f_..."]

}
```

Start response shape:

```
{

  "session_id": "uuid",

  "status": "running",

  "engine": "computer_use",
```



```
"provider": "google|anthropic|openai"

}
```

Flow B: Session loop and provider execution

1. `AgentLoop.run()` **sets status to** `RUNNING`.
2. `AgentLoop._run_computer_use_engine()` **maps provider and builds prompt via** `backend/prompts.py::get_system_prompt(...)`.
3. `ComputerUseEngine.execute_task()` **delegates to** `backend/providers/run_client(...)`.
4. **Provider wrapper** (`backend/providers/openai.py`, `anthropic.py`, `gemini.py`) **can run** `maybe_plan_with_web_search(...)` **first**.
5. **Provider client emits turn records/logs**; `AgentLoop._on_turn()` **maps action data to** `StepRecord`.
6. **Backend broadcasts step events to** `WebSocket` **clients**.
7. **Session ends as** `COMPLETED`, `STOPPED`, **or** `ERROR`; **backend broadcasts** `agent_finished`.

Step record shape:

```
{

  "step_number": 1,

  "timestamp": "ISO-8601",

  "screenshot_b64": "base64 png|null",

  "action": {

    "action": "ActionType",

    "coordinates": [x, y],

    "text": "optional",

    "reasoning": "optional"

  },

}
```

```
"raw_model_response": "optional",

"error": "optional"

}
```

Flow C: Tool action translation to desktop operations

1. Provider action reaches `DesktopExecutor.execute(name, args)`.
2. Executor maps to handlers like `_act_click_at`, `_act_type_text_at`, `_act_scroll_at`.
3. Handler posts `/action` to in-container service.
4. `docker/agent_service.py::AgentHandler.do_POST()` validates auth and action allowlist, then dispatches via `_dispatch_action()` and `_dispatch_desktop()`.
5. `xdotool/scrot` operations run and return structured result.
6. Executor wraps result as `CUActionResult` and returns it upstream.

Example `/action` payload:

```
{

  "action": "type_text_at",

  "coordinates": [640, 360],

  "text": "hello world",

  "press_enter": true,

  "clear_before": true,

  "mode": "desktop",

  "action_id": "uuid:0",

  "include_screenshot": 1

}
```

Flow D: WebSocket stream and screenshot mode

1. Frontend opens `/ws` in `useWebSocket.connect()`.
2. Client sends heartbeat pings every 15 seconds.
3. Client sends `screenshot_mode` messages based on `ScreenView` mode (interactive noVNC vs screenshot fallback).
4. Backend validates origin/token and manages subscriber sets.
5. Backend emits events validated by `backend/server/ws_schema.py`.

Consumed events: - `screenshot` - `screenshot_stream` - `log` - `step` - `agent_finished` - `pong`

Flow E: File upload and provider-specific attachment handling

1. `ControlPanelView` uploads files via `uploadFile(file)` to `POST /api/files/upload`.
2. Backend streams files into `FileStore.add_stream()`.
3. Response includes `file_id`; frontend sends these in `attached_files` during start.
4. `validate_attached_files(provider, file_ids)` enforces format/existence/dedup and provider compatibility.
5. Provider paths:
6. OpenAI: `prepare_openai_file_search()` creates vector store and uploads files.
7. Anthropic: `prepare_anthropic_documents()` uploads PDF/TXT as Files API docs; MD/DOCX are extracted and inlined.
8. Gemini: rejected for CU with files (`GEMINI_CU_FILE_REJECTION`).

Upload response shape:

```
{

  "file_id": "f_...",

  "filename": "doc.pdf",

  "size_bytes": 12345,

  "mime_type": "application/pdf"

}
```

Flow F: Safety confirmation handshake

1. Provider requests confirmation; `AgentLoop._on_safety()` calls `safety_registry.arm(session_id)`.
2. Backend emits a log event with `type: "safety_confirmation"`, `session_id`, `nonce`, and `explanation`.
3. `SafetyModal` calls `confirmSafety(sessionId, confirm, nonce)`.
4. Backend verifies nonce with `safety_registry.verify_nonce(...)`, sets decision, and unblocks the loop.
5. Timeout path auto-denies.

Module 4: Setup and Run Guide

Prerequisites

- Docker daemon and Docker Compose.
- Python 3.11+ (checked by setup scripts).
- Node.js (for Vite/React frontend).

Environment setup

1. Copy `.env.example` to `.env`.
2. Set required compose keys:
3. `AGENT_SERVICE_TOKEN=...`
4. `VNC_PASSWORD=...`
5. Set at least one provider API key:
6. `GOOGLE_API_KEY` or `GEMINI_API_KEY`
7. `ANTHROPIC_API_KEY`
8. `OPENAI_API_KEY`
9. Optional:
10. `OPENAI_BASE_URL`
11. `HOST`, `PORT`
12. `CUA_WS_TOKEN` and `CUA_ALLOW_PUBLIC_BIND=1` for external binding
13. `CORS_ORIGINS`, `CUA_ALLOWED_HOSTS`, `CUA_MAX_BODY_BYTES`

Typical command sequences

Path A: one-command bootstrap and launch

```
bash setup.sh
```

Path B: manual daily startup

```
uv sync

cd frontend && npm install && cd ..

docker compose up -d

python dev.py
```

Runtime access

- UI: `http://localhost:3000`
- Backend default port: 8100
- API docs via proxied `/docs`
- noVNC via `/vnc/vnc.html?...`

Database/external seeding

- No database migration or seeding step exists in this repository.
- External runtime dependency is the Dockerized desktop service plus upstream provider APIs.

PDF export command

```
pandoc -f gfm -t pdf docs/zero-to-hero-study-handbook.md -o docs/zero-to-hero-study-handbook.pdf
```

Module 5: Study Plan and Practice Exercises

Ordered study plan

1. Read `backend/models/schemas.py` to lock in data contracts.
2. Read `backend/server/__init__.py`, especially `/api/agent/start`, `/ws`, `/api/files/upload`.
3. Read `backend/loop.py` for session lifecycle.
4. Read `backend/engine/__init__.py` and one provider file (`backend/engine/openai.py`).
5. Read `backend/executor.py` and `docker/agent_service.py` together.
6. Read `frontend/src/api.js`, `useSessionController.js`, and

`useWebSocket.js`.

7. Read `frontend/src/pages/WorkbenchPage.jsx` and `ControlPanelView.jsx`.
8. Read infra files: `backend/infra/config.py`, `backend/infra/docker.py`, `.env.example`, `docker-compose.yml`.
9. Review tests: `tests/test_server.py`, `tests/test_provider_run_contract.py`, `tests/docker/test_agent_service.py`.
10. Revisit `backend/providers/planner.py`, `backend/files.py`, and `backend/infra/storage.py`.

Practice exercises with model answer outlines

1. Trace how `useBuiltinSearch` changes behavior end-to-end.
2. Files: `WorkbenchPage.jsx`, `api.js`, `backend/server/__init__.py`, `backend/providers/_common.py`, `backend/providers/planner.py`.
3. Outline: UI toggle sets payload key; backend validates; provider wrapper runs planning; planner brief is appended into CU task text.
4. Explain why `/api/agent/start` can return 409 even when container exists.
5. Files: `backend/server/__init__.py`, `backend/infra/docker.py`.
6. Outline: container process can exist while agent service is `unready`; backend re-checks readiness state before session creation.
7. Map one `click_at` action from model output to final execution.
8. Files: `backend/loop.py`, `backend/executor.py`, `docker/agent_service.py`.
9. Outline: turn action mapped to executor handler; executor posts `/action`; agent service dispatches to `_xdo_click`.
10. Explain the safety nonce handshake and anti-replay guard.
11. Files: `backend/loop.py`, `backend/safety.py`, `SafetyModal.jsx`, `api.js`, `backend/server/__init__.py`.
12. Outline: backend arms nonce and event, frontend echoes nonce, backend verifies nonce before setting decision.
13. Show how unsupported provider/model combinations are blocked.
14. Files: `backend/models/allowed_models.json`, `backend/server/__init__.py`, `WorkbenchPage.jsx`.

15. Outline: frontend filters model dropdown; backend enforces allowlist and rejects invalid combos.
16. Explain screenshot-load optimization when noVNC is active.
17. Files: `ScreenView.jsx`, `useWebSocket.js`,
`backend/server/__init__.py`.
18. Outline: UI switches screenshot mode off in interactive mode, backend unsubscribes stream demand and can suspend publisher loop.
19. Compare OpenAI vs Anthropic file attachment ingestion.
20. Files: `backend/files.py`, `backend/engine/openai.py`,
`backend/engine/claude.py`.
21. Outline: OpenAI uses vector store/file search; Anthropic uses Files API docs for PDF/TXT and inline extraction for MD/DOCX.
22. List three backend hardening controls.
23. Files: `backend/main.py`, `backend/server/__init__.py`, `.env.example`.
24. Outline: external bind guardrail; host allowlist middleware; token-gated WS/REST mutation routes.

Understanding Verification Checklist

- Can you explain `POST /api/agent/start` from UI payload to `AgentLoop` creation?
- Can you explain how `AgentLoop` transforms provider turns into `StepRecord` timeline events?
- Can you map provider tool actions to `DesktopExecutor` and then to `_dispatch_desktop()` in `docker/agent_service.py`?
- Can you explain WS event production and consumption (`/ws`, `validate_outbound`, `useWebSocket`)?
- Can you explain provider-specific file handling and why Gemini rejects `attached_files` for CU runs?
- Can you explain the safety nonce handshake and timeout behavior end-to-end?
- Can you list required `.env` keys for a secure local compose path?
- Can you explain where and why 400, 401, 403, 409, 429, and 503 appear in core flows?
- Can you identify where model allowlists and capability constraints are defined and enforced?
- Can you narrate the architecture from `backend/main.py` to frontend

completion events without skipping major modules?